Guía del código del Generador de EMOM

JavaScript explicado línea por línea, para principiantes

Este documento explica cada parte de tu archivo JavaScript. Está pensado para estudiar: primero tienes la idea general, luego un pequeño diccionario de palabras clave y después el código troceado en tablas, con el código a la izquierda y su significado a la derecha.

1. ¿Qué hace este programa?

Tienes un formulario donde eliges un ejercicio y escribes tu **RM** (el peso máximo que puedes levantar una vez). Al pulsar el botón, el programa dibuja una tabla de **14 minutos** con las repeticiones y el peso que te toca en cada tramo, calculado como un porcentaje de tu RM.

El código hace tres cosas:

- **Validar:** si el RM es demasiado bajo o demasiado alto, muestra un mensaje de error propio.
- **Calcular:** para cada tramo multiplica tu RM por un porcentaje y lo redondea a discos de 2,5 kg.
- **Mostrar:** crea las filas de la tabla y hace visibles el resultado y el enlace.

> **Aviso sobre los textos:** Algunos mensajes de error y consejos de la tabla de tu código contienen insultos y expresiones despectivas (incluida una contra el colectivo LGTBI). Por eso en este PDF aparecen abreviados como `"..."`. Te recomiendo sustituirlos por frases neutras o de ánimo, sobre todo si vas a entregar o publicar el proyecto. El código funciona exactamente igual con otro texto.

2. El recorrido completo, paso a paso

Paso	Qué pasa	Qué parte del código lo hace
1	La persona escribe su RM en el campo.	El primer bloque (evento `input`)
2	Si el número es muy bajo o muy alto, aparece un error propio.	`setCustomValidity`
3	La persona pulsa el botón del formulario.	El evento `submit`
4	El código frena la recarga de la página y lee los datos.	`e.preventDefault()` y `.value`
5	Recorre las 7 fases y calcula el peso de cada una.	`FASES.forEach(...)`
6	Crea una fila por fase y la mete en la tabla.	`createElement` y `appendChild`
7	Hace visibles el resultado, la tabla y el enlace.	`hidden = false`

3. Mini diccionario antes de empezar

Estas palabras salen una y otra vez. Si las entiendes, el resto es fácil.

Palabra	Significado sencillo	Ejemplo
const	Crea una variable que no vas a reasignar. Es como una caja con etiqueta donde guardas un dato.	`const edad = 20;`
DOM	La página HTML convertida en objetos que JavaScript puede leer y modificar.	—
getElementById	Busca en la página el elemento que tiene ese `id` y te lo da.	`document.getElementById("rm")`
querySelector	Como el anterior, pero buscas con selectores de CSS (#id, .clase, etiqueta).	`document.querySelector("#tablaEmom tbody")`
Evento	Algo que ocurre en la página: escribir, hacer clic, enviar un formulario.	`input`, `submit`, `click`
addEventListener	"Escucha" un evento y ejecuta una función cuando ocurre.	`boton.addEventListener("click", ...)`
Función flecha	Una función escrita con `=>`. Es un bloque de código que se ejecuta cuando lo llaman.	`() => { ... }`
Array	Una lista ordenada de cosas, entre corchetes.	`[10, 20, 30]`
Objeto	Un grupo de datos con nombre (propiedad: valor), entre llaves.	`{ reps: 5, porcentaje: 65 }`
Template string	Texto entre comillas invertidas donde `${...}` se sustituye por el valor de una variable.	`` `Hola ${nombre}` ``
textContent	El texto de un elemento. Puedes leerlo o cambiarlo.	`titulo.textContent = "Hola"`
innerHTML	El HTML que hay dentro de un elemento. Puedes leerlo o reemplazarlo.	`tbody.innerHTML = ""`

4. Parte A: el mensaje de error del campo RM

Este bloque personaliza el aviso que muestra el navegador cuando el RM está fuera de rango. Para que funcione, el `<input>` del HTML debe tener los atributos `min` y `max`.

CÓDIGO	¿QUÉ SIGNIFICA?
`// MENSAJE DE ERROR PERSONALIZADO` `// PARA EL INPUT DEL RM`	Los comentarios empiezan con `//`. El navegador los ignora; sirven para que tú (o quien lea) entiendas el código.
`const rm = document.getElementById("rm");`	Busca el elemento con `id="rm"` (el campo donde se escribe el RM) y lo guarda en la variable `rm`. Desde ahora `rm` representa ese campo.
`rm.addEventListener("input", () => {`	Dice: "cada vez que cambie lo escrito en este campo (evento `input`), ejecuta lo que hay entre las llaves". Se ejecuta con cada tecla pulsada.
`    rm.setCustomValidity("");`	**Limpia** el mensaje de error anterior. Si no lo hicieras, el campo seguiría marcado como inválido aunque corrigieras el número.
`    if (rm.validity.rangeUnderflow) {`	`rm.validity` es el "informe de estado" del campo. `rangeUnderflow` vale `true` cuando el número es **menor** que el `min` del HTML.
`        rm.setCustomValidity("...");`	Pone un mensaje de error propio (aquí abreviado). Mientras exista ese mensaje, el navegador **no deja enviar** el formulario y lo muestra.
`    } else if (rm.validity.rangeOverflow) {`	Si no fue demasiado bajo, comprueba lo contrario: `rangeOverflow` es `true` cuando el número es **mayor** que el `max`.
`        rm.setCustomValidity("...");`	Mensaje propio para el caso de número demasiado alto (aquí abreviado).
`    }` `});`	Cierran el `if` y la función flecha. `});` termina también la llamada a `addEventListener`.

Idea clave: `setCustomValidity("")` con texto vacío significa "el campo es válido". Con cualquier texto significa "el campo es inválido y este es el error".

5. Parte B: guardar los elementos de la página

Antes de trabajar, el código "agarra" los elementos que va a necesitar y los guarda en variables. Así no tiene que buscarlos cada vez.

CÓDIGO	¿QUÉ SIGNIFICA?
`const form = document.getElementById("emomForm");`	Guarda el formulario completo (el elemento con `id="emomForm"`).
`const resultado = document.getElementById("resultado");`	Guarda el contenedor que se mostrará con el resultado.

CÓDIGO	¿QUÉ SIGNIFICA?
`const resultadoTexto = document.getElementById("resultadoTexto");`	Guarda el texto-título del resultado, por ejemplo "EMOM de 14 minutos: ...".
`const tbody = document.querySelector("#tablaEmom tbody");`	Busca con selector CSS: el `<tbody>` que está dentro del elemento `#tablaEmom`. Es el cuerpo de la tabla, donde se añadirán las filas.

> **Fíjate:** Cada `id` debe estar escrito **exactamente igual** en el HTML y en el JavaScript, incluidas mayúsculas. Si no existe, el resultado es `null` y verás un error en la consola.

6. Parte C: la lista de fases (FASES)

`FASES` es un **array** (lista) con **7 objetos**. Cada objeto describe un tramo de 2 minutos del EMOM. Se escribe en mayúsculas por convención, para indicar que es una constante que no cambia.

```
const FASES = [
    { minutos: "Min 1 - 2", porcentaje: 65, reps: 5, enfoque: "..." },
    { minutos: "Min 3 - 4", porcentaje: 70, reps: 4, enfoque: "..." },
    ...
    { minutos: "Min 13 - 14", porcentaje: 95, reps: 1, enfoque: "..." }
];
```

Propiedad	Qué guarda	Tipo
`minutos`	Etiqueta del tramo que se muestra en la tabla.	Texto
`porcentaje`	Qué porcentaje de tu RM debes levantar en ese tramo.	Número
`reps`	Cuántas repeticiones hacer cada minuto.	Número
`enfoque`	Consejo o comentario del tramo (aquí abreviado: contiene insultos).	Texto

Resumen de las 7 fases (el texto de `enfoque` se omite):

Minutos	% del RM	Reps
1 - 2	65 %	5
3 - 4	70 %	4
5 - 6	75 %	3
7 - 8	80 %	2
9 - 10	85 %	1
11 - 12	90 %	1

Minutos	% del RM	Reps
13 - 14	95 %	1

> **Para acceder a un dato:** Con un objeto `fase` escribes `fase.porcentaje` (punto + nombre de la propiedad) y obtienes, por ejemplo, `65`.

7. Parte D: qué pasa al enviar el formulario

Este es el bloque más importante. Se ejecuta cuando alguien pulsa el botón de enviar. Lo dividimos en cuatro tablas.

7.1 Empezar y leer los datos

CÓDIGO	¿QUÉ SIGNIFICA?
`form.addEventListener("submit", (e) => {`	Escucha el evento `submit` (enviar formulario). `e` es el "objeto evento": contiene información de lo que ha ocurrido.
`e.preventDefault();`	Los formularios, por defecto, recargan la página al enviarse. Esto **lo cancela** para que JavaScript controle todo sin recargar.
`const select = document.getElementById("ejercicio");`	Guarda el desplegable (`<select>`) donde se elige el ejercicio.
`const ejercicio = select.options[ select.selectedIndex].text;`	`select.options` es la lista de opciones. `selectedIndex` es la posición de la elegida (0, 1, 2...). `.text` es lo que **se ve** en pantalla (distinto de `.value`).
`const rmKg = Number( document.getElementById("rm").value);`	`.value` siempre devuelve **texto**, aunque escribas un número. `Number(...)` lo convierte en número real para poder multiplicar.

7.2 Preparar el título y vaciar la tabla

CÓDIGO	¿QUÉ SIGNIFICA?
`resultadoTexto.textContent = \`EMOM de 14 minutos: ${ejercicio} (RM: ${rmKg} kg)\`;`	Cambia el texto del título. Las **comillas invertidas** permiten un template string: `${ejercicio}` y `${rmKg}` se sustituyen por sus valores. Ejemplo: "EMOM de 14 minutos: Sentadilla (RM: 100 kg)".
`tbody.innerHTML = "";`	**Vacía** el cuerpo de la tabla. Sin esto, cada vez que envías el formulario se añadirían filas debajo de las anteriores.

7.3 Recorrer las fases y construir la tabla

CÓDIGO	¿QUÉ SIGNIFICA?
`FASES.forEach((fase) => {`	`forEach` recorre la lista. Ejecuta el bloque **una vez por cada fase**. En cada vuelta, `fase` es un objeto distinto (la 1ª, la 2ª...).
`const peso = Math.round( (rmKg * fase.porcentaje / 100) / 2.5 ) * 2.5;`	Calcula el peso: `rmKg * fase.porcentaje / 100` es el porcentaje del RM. Luego se divide entre 2,5, se redondea con `Math.round` y se multiplica por 2,5. Resultado: el **múltiplo de 2,5 kg más cercano**, que es lo que se puede cargar con discos.
`const fila = document.createElement("tr");`	Crea una fila de tabla (`<tr>`) nueva. Existe solo en memoria: todavía no se ve.
`fila.innerHTML = ` <td>${fase.minutos}</td> <td>${fase.reps} ${fase.reps === 1 ? "Rep" : "Reps"}</td> <td>${peso} kg</td> <td>${fase.enfoque}</td> `;`	Rellena la fila con cuatro celdas (`<td>`): minutos, repeticiones, peso y enfoque. La parte `fase.reps === 1 ? "Rep" : "Reps"` es un **operador ternario**: "si reps es igual a 1, escribe Rep; si no, escribe Reps". Así sale "1 Rep" pero "5 Reps". `===` compara valor y tipo.
`tbody.appendChild(fila);`	**Inserta** la fila dentro del cuerpo de la tabla. En este momento aparece en pantalla.
`});`	Cierra el `forEach`. Se repite hasta acabar las 7 fases.

7.4 Mostrar todo

CÓDIGO	¿QUÉ SIGNIFICA?
`resultado.hidden = false;`	El atributo `hidden` oculta un elemento. Al ponerlo en `false` el contenedor del resultado se hace visible.
`document.getElementById("tablaEmom") .hidden = false;`	Hace visible la tabla (en el HTML empieza oculta con `hidden`).
`document.getElementById("linkhidden") .hidden = false;`	Hace visible el enlace a la imagen del EMOM.
`});`	Cierra la función del evento `submit` y la llamada a `addEventListener`.

8. Ejemplo con números reales

Supón que escribes **RM = 87 kg**. Vamos a calcular el peso de la primera fase (65 %):

Paso	Operación	Resultado
1	87 × 65 / 100	56,55
2	56,55 / 2,5	22,62
3	Math.round(22,62)	23
4	23 × 2,5	**57,5 kg**

Con **RM = 100 kg** todo da múltiplos exactos de 2,5, así que la tabla sale así:

Minutos	Reps	Peso
1 - 2	5 Reps	65 kg
3 - 4	4 Reps	70 kg
5 - 6	3 Reps	75 kg
7 - 8	2 Reps	80 kg
9 - 10	1 Rep	85 kg
11 - 12	1 Rep	90 kg
13 - 14	1 Rep	95 kg

9. El HTML que necesita este código

El JavaScript solo funciona si el HTML tiene estos elementos con estos `id`. Es un esqueleto mínimo de referencia:

```
<form id="emomForm">
  <select id="ejercicio">
    <option value="sentadilla">Sentadilla</option>
  </select>
  <input id="rm" type="number" min="20" max="300" required>
  <button type="submit">Generar</button>
</form>

<div id="resultado" hidden>
  <p id="resultadoTexto"></p>
  <table id="tablaEmom" hidden>
    <thead>...</thead>
    <tbody></tbody>
  </table>
  <a id="linkhidden" href="emom.png" hidden>Ver imagen</a>
</div>
```

Nota: Los valores `min="20"` y `max="300"` son solo un ejemplo; usa los de tu proyecto. Sin `min` y `max`, los avisos de `rangeUnderflow` y `rangeOverflow` nunca saltan.

10. Errores típicos de principiante

Error	Síntoma	Solución
Escribir mal un `id`	La consola muestra "Cannot read properties of null".	Comparar el `id` del HTML con el del JS, letra por letra.
Olvidar `Number()`	Los cálculos fallan o se concatenan como texto.	Convertir siempre `.value` antes de calcular.
Quitar `preventDefault()`	La página se recarga y la tabla desaparece.	Mantenerlo como primera línea del `submit`.
Olvidar `tbody.innerHTML = ""`	Las filas se acumulan en cada envío.	Vaciar la tabla antes de rellenarla.
No limpiar `setCustomValidity("")`	El campo queda inválido para siempre.	Limpiar al inicio del evento `input`.
Cargar el script antes del HTML	Los elementos salen como `null`.	Poner `<script>` al final del `<body>` o usar `defer`.

11. Mini test de repaso

Intenta responder sin mirar. Las soluciones están justo debajo.

1. ¿Para qué sirve `e.preventDefault()`?
2. ¿Por qué hay que usar `Number()` con el valor del input?
3. ¿Qué hace `Math.round(x / 2.5) * 2.5`?
4. ¿Qué diferencia hay entre `textContent` e `innerHTML`?
5. ¿Qué hace `tbody.appendChild(fila)`?
6. ¿Qué pasa si `hidden` se pone en `false`?

Soluciones

1	Evita que el navegador recargue la página al enviar el formulario, para que JavaScript controle el proceso.
2	Porque `.value` devuelve texto. Con `Number()` se convierte en número y se puede multiplicar y redondear.
3	Redondea `x` al múltiplo de 2,5 más cercano.
4	`textContent` trabaja con texto plano. `innerHTML` interpreta etiquetas HTML y puede crear elementos.
5	Inserta la fila creada dentro del cuerpo de la tabla, haciéndola visible.
6	El elemento deja de estar oculto y se muestra.